



FOR JAVA DEVELOPERS MOVING TO NESTJS

Rolled back. Receipt sent.

Spring Boot to NestJS, episode 29: application events

An application event is still part of your execution model. Publishing inside a transaction does not make the event transactional.

A declined payment rolled back. A plain Spring `@EventListener` and a Nest `@OnEvent` had both already sent the receipt.

Spring's `@TransactionalEventListener` did not run on rollback, and ran after the commit. Nest has no equivalent here: emit after the commit yourself.

A throwing Spring listener reached the publisher. A throwing Nest listener was caught and logged, unless `suppressErrors` was off, when an async one took the process down.

In scope

- `ApplicationEventPublisher` and `EventEmitter2`
- Listener failures
- Who waits: `emit`, `emitAsync`, `@Async`
- Events and transactions

Not in scope

- The outbox pattern in depth
- Events between services

At the time of recording, `@OnEvent` listeners suppress errors by default.

01 The measurements

SPRING publishEvent, LISTENER THROWS	threw IllegalStateException
NEST emit, LISTENER THROWS	did not throw; error logged
NEST suppressErrors false, ASYNC	process exited, code 1, at 18 ms
SPRING publish / @Async	303 ms / 1 ms
NEST emit / emitAsync	7 ms (listener ran to 307) / 302 ms
ROLLED BACK, PLAIN LISTENER, BOTH	payments 0, receipts 1
@TransactionalEventListener / emit after COMMIT	rolled back 0; committed 1

A 300 ms listener for timing; a listener that throws for failure.



02 Emit after the commit

@OnEvent runs while your transaction is still open. Commit first, then emit. If the event must survive a crash between the commit and the emit, write it to an outbox table in the same transaction.

payments.service.ts

```
await this.dataSource.transaction(async (manager) => {
  await manager.save(Payment, payment);
});
this.events.emit('payment.created', payment.id);
```

emit only after COMMIT: rolled back, receipts sent 0; committed, receipts sent 1

Run on a real machine, 2026-10-01, by scripts/verify-events.sh in the course repository.

03 Hear the failure

emit() never hands a listener's error back. Only emitAsync with suppressErrors: false on the listener rejects in the publisher; suppressErrors: false with plain emit crashed the process.

receipt.listener.ts

```
@OnEvent('payment.created', { suppressErrors: false })
async sendReceipt(id: string) { /* ... */ }

await this.events.emitAsync('payment.created', id); // rejects if it throws
```

emitAsync with suppressErrors false: rejected: mail server busy after 501 ms

Run on a real machine, 2026-10-01, by scripts/verify-events.sh in the course repository.

04 The Spring side

@TransactionalEventListener runs after commit by default and not at all on rollback. A plain @EventListener runs on the publisher's thread, inside the transaction.

ReceiptListener.java

```
@TransactionalEventListener
void receiptAfterCommit(PaymentEvents.Created event) {
```

rolled back: @EventListener receipts 1, @TransactionalEventListener receipts 0

Run on a real machine, 2026-10-01, by scripts/verify-events.sh in the course repository.

Before you trust an application event

1. Does the publisher wait for the listener?
2. Does a listener failure reach the publisher, get logged, or crash the process?
3. Is the event emitted inside a transaction?
4. Can the listener act on work that later rolls back?
5. Must the event survive a crash? Then it needs an outbox.

Everything else

 Site https://code-with-sam-dev.github.io	 Facebook https://www.facebook.com/codewithsamdev
 YouTube https://www.youtube.com/@CodewithSam-Dev	 GitHub https://github.com/code-with-sam-dev
 LinkedIn https://www.linkedin.com/company/code-with-sam-dev	 Buy me a coffee https://buymeacoffee.com/codewithsam
 X https://x.com/CodeWithSamDev	 Buy me sadza https://buymesadza.com/codewithsam
 TikTok https://www.tiktok.com/@codewithsamdev	Every measurement, one command: scripts/verify-events.sh https://github.com/code-with-sam-dev/spring-to-node
 Instagram https://www.instagram.com/codewithsamdev	NestJS events https://docs.nestjs.com/techniques/events

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Spring is a trademark of Broadcom. Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-10-01 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.